Université du Luxembourg

Introduction to Message Passing Interface (MPI)

Lesson 4

Matteo Barborini

University of Luxembourg - HPC Platform

matteo.barborini@uni.lu

7th of October 2026



Université du Luxembourg

Exercise: Matrix Vector multiplication (dgemv)



Parallelization of dgemv with `MPI_Send` and `MPI_Recv`

Exercise: Matrix Vector multiplication (dgemv)

We consider a matrix $\mathbf{A}$ with *N* rows and *M* columns, and a vector $\mathbf{b}$ of length *M*, both containing double precision elements.

We define the matrix vector product between the transposed of $\mathbf{A}$ and the vector $\mathbf{b}$ as $\mathbf{c} = \mathbf{A}\mathbf{b}$ and the single element of the $\mathbf{c}$ will be given by

$$\mathbf{c}_i = \sum_{j=1}^{N} \mathbf{A}_{ij}\mathbf{b}_j$$

Exercise: Write a serial code to do a Matrix vector operation, and then parallelize it with point-to-point communications MPI_Send and MPI_Recv.

1. Rank 0 Initializes matrix $\mathbf{A}$ and vector $\mathbf{b}$.

2. Rank 0 communicates to all other ranks the vector $\mathbf{b}$ and parts of the matrix $\mathbf{A}$ necessary for the local matrix vector multiplication.

4. Each process executes the local matrix vector product (Local partial segments of vector $\mathbf{c}$).

5. Rank 0 collects all the partial segments of the vector $\mathbf{c}$ from each process

6. Rank 0 prints vector $\mathbf{c}$.

Matrix allocations in C

Exercise: Matrix Vector multiplication (dgemv)

Examples of matrix allocations in C can be found in the example `4.1.MatrixAllocations.c`

The simplest way to allocate matrices in C is to first allocate a vector of pointers and then for each pointer a vector of length column.

```c
// Allocate empty matrix
double **allocate_2d_double(int rows, int columns)
{
    if (rows <= 0 || columns <= 0)
        return NULL;

    // Allocate memory for the row pointers (an array of double*)
    double **matrix = (double **)malloc(rows * sizeof(double *));

    // Allocate memory for the elements of each row (and initialize to 0)
    for (int i = 0; i < rows; i++)
    {
        matrix[i] = (double *)calloc(columns, sizeof(double));
    }
    return matrix;
}
```

The memory is not necessarily contiguous!! What can happen in MPI transfers of non-contiguous data?

Matrix allocations in C

Exercise: Matrix Vector multiplication (dgemv)

An alternative way to allocate contiguous memory for a matrix is to allocate a single vector of length rows*columns.

If we define the function

```c
// Allocate empty vector
double *allocate_1d_double(int elements)
{
    if (elements <= 0)
        return NULL;

    // Allocate memory for the pointers (an array of double*)
    double *vector = (double *)calloc(elements, sizeof(double));
    return vector;
}
```

We simply allocate a vector of length rows*columns

```c
vector = allocate_1d_double(rows * columns);
```

The element i, j of the matrix can then be accessed as `vector[i*columns + j]` assuming the allocation is again "columns first".

Remember that in Fortran it is the opposite, Fortran uses "rows first" allocation.

Matrix allocations in C

Exercise: Matrix Vector multiplication (dgemv)

An alternative way to allocate contiguous memory for a matrix is to allocate a single temporary vector of length `rows * columns`.

Afterwards, the matrix can be allocated and the pointers can be associated to the ones used for the larger vector.

```c
// Allocate empty matrix (consecutive elements)
double **allocate_2d_double_blocked(int rows, int columns)
{
    if (rows <= 0 || columns <= 0)
        return NULL;

    /* allocate the n*m contiguous items */
    double *temp_matrix = (double *)malloc(rows * columns * sizeof(double));

    /* allocate the row pointers into the memory */
    double **matrix = (double **)malloc(rows * sizeof(double *));

    /* set up the pointers into the contiguous memory */
    for (int i = 0; i < rows; i++)
        matrix[i] = &(temp_matrix[i * columns]);

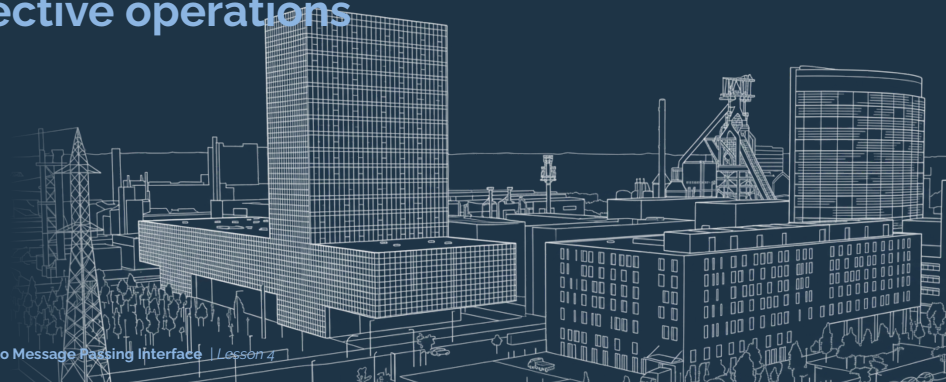
    free(temp_matrix);

    return matrix;
}
```

In this way the memory is contiguous and we can use the matrix as a 2D array.

Université du Luxembourg

MPI Collective operations



Point-to-point versus Collective Communication

MPI Collective operations

Point-to-Point communications:

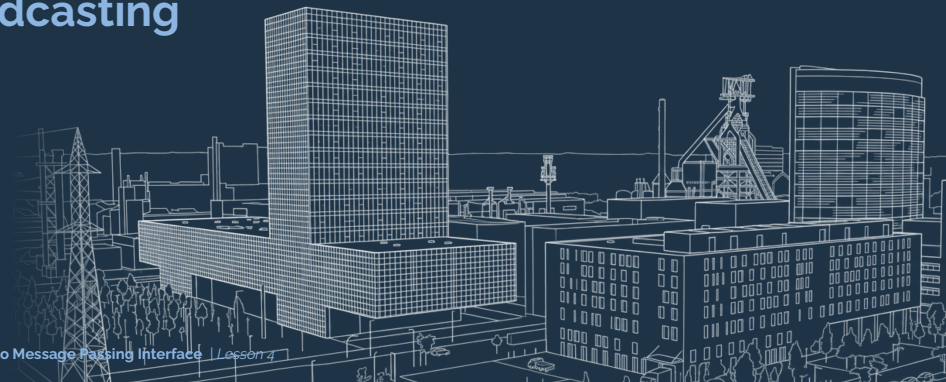
- Point-to-Point communications involve two specific processes.
- Within the application one needs to specify the source and the destination of the communication (`MPI_*Send`, `MPI_*Recv`).

Collective communications:

- The communication functions are called synchronously by **all MPI processes** in the communicator.
- The functions are **Blocked** until the collective operation requirements are met.
- Message passing is optimized internally through tree-based or hypercube topologies ($\mathcal{O}(\log_2 p)$ scaling, where p is the number of processes).
- No tags are necessary since the functions strictly match to order of invocation.

Université du Luxembourg

MPI broadcasting



MPI_Bcast function

MPI broadcasting

The Bcast function is used to broadcast data, of type **MPI_datatype** to all participants of the **MPI_COMM_WORLD** environment

```
int MPI_Bcast( void *data,
               int data_size,
               MPI_Datatype data_type,
               int bcasting_rank,
               MPI_Comm comm )
```

```
int MPI_Ibcast( void *data,
                int data_size,
                MPI_Datatype data_type,
                int bcasting_rank,
                MPI_Comm comm )
                MPI_Request *request )
```

Properties:

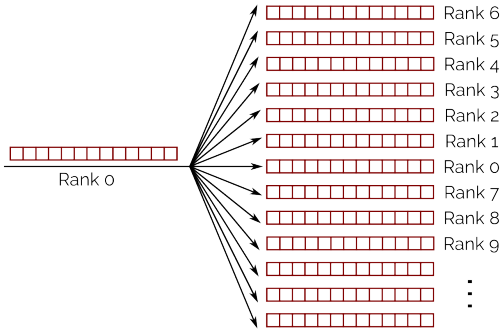
- **MPI_Bcast** is a blocking function, while **MPI_Ibcast** is non-blocking;
- The data is sent from the process with rank **bcasting_rank** to all other processes in the communicator;
- All processes must call this function, otherwise the program will hang.

The use cases of the Bcast function are:

- Broadcasting input variables of a simulation;
- Broadcasting the results of a computation;
- Broadcasting information about the state of the simulation;
- Broadcasting information about the state of the parameters of the parallelization of the simulation.

MPI_Bcast function

MPI broadcasting



In Bcasting a vector the receiving ranks must know the size of the data to be transfered and the data type.

Example of usage of MPI_Bcast function

MPI broadcasting

A complete example of the usage of the MPI_Bcast function can be found in the example `4.1.MPI_Bcast.c` file.

In the example we are Bcasting a vector of **`number_of_elements`** elements from rank 0 to all other ranks.

At the beginning of the program, rank 0 defines the number of elements of the vector and then broadcasts it to all other ranks.

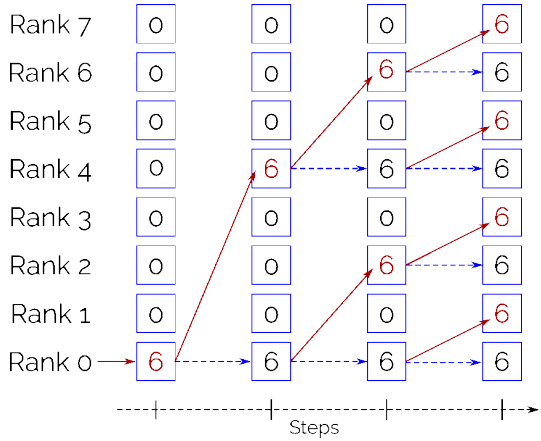
```
1   int number_of_elements;
2   double *vector;
3
4   // Rank 0 defines the number of elements of the vector
5   if (mpi_rank == 0)
6   {
7      number_of_elements = 100;
8   }
9   // The number of elements is broadcasted to all ranks
10  MPI_Bcast(&number_of_elements, 1, MPI_INT, 0, MPI_COMM_WORLD);
11
12  // All ranks allocate memory for the vector
13  vector = (double *)calloc(number_of_elements, sizeof(double));
```

Before broadcasting the vector all rank allocate it.

Tree algorithms for collective communications

MPI broadcasting

Efficient collective communication algorithms are based on the use of tree structures, that reduce the computational cost to $\mathcal{O}(\log_2(p))$ where p is the number of processes.



Université du Luxembourg

MPI Reduction Operations



MPI_Reduce function

MPI Reduction Operations

It is collective MPI communication function that combines data provided by all processes in a communicator using a specified reduction operation (such as addition, maximum, minimum, or logical AND) and yields a single reduced result on a specified root process.

```
int MPI_Reduce(
            const void *
    data_to_reduce,
            void *data_reduced,
            int data_size,
            MPI_Datatype data_type,
            MPI_Op reduce_operation,
            int receiving_rank,
            MPI_Comm comm)
```

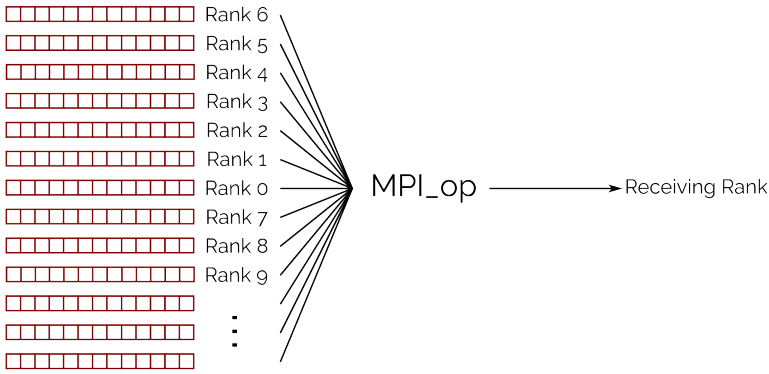
```
int MPI_Ireduce(const void *data_to_reduce,
            void *data_reduced,
            int data_size,
            MPI_Datatype data_type,
            MPI_Op reduce_operation,
            int receiving_rank,
            MPI_Comm comm,
            MPI_Request *request)
```

Characteristics:

- Every process in the communicator (comm) must call MPI_Reduce with matching parameters (`data_size`, `data_type`, `reduce_operation`, `receiving_rank`);
- The receive buffer (`data_reduced`) variable is only meaningful on the process specified as root (`receiving_rank`);
- Rather than gathering all data sequentially on process 0, MPI implementations typically use tree-based or ring-based structures internally, achieving $\mathcal{O}(\log_2 p)$ time complexity for p processes

MPI_Reduce function

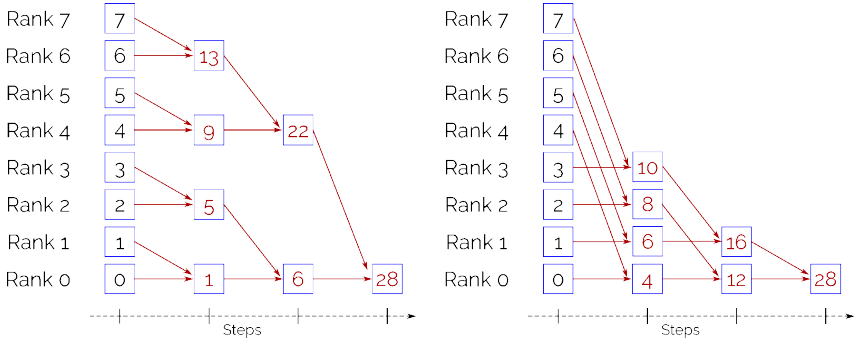
MPI Reduction Operations



Tree algorithms for Reduce operations

MPI Reduction Operations

Efficient collective communication algorithms are based on the use of tree structures, that reduce the computational cost to $\mathcal{O}(\log_2 p)$ where p is the number of processes.



Notice that the choice of the tree depends on the number of processes, the specific reduction operation, the underlying hardware architecture, and the MPI implementation.

MPI_Reduce function

MPI Reduction Operations

A complete example of the usage of the MPI_Reduce function can be found in the example `4.2.MPI_Reduce.c` file.

After initializing the MPI environment, all ranks initialize a local variable `local_values_to_sum` with their rank value, and the variable containing the result of the reduction operation is initialized to 0.

```
1   // All ranks initialize the values that contribute to the sum.
2   double local_values_to_sum = (double)mpi_rank;
3
4   // All ranks initialize the values of the sum
5   double sum = 0.0;
6
7   // Perform the reduction operation to sum values across all ranks
8   MPI_Reduce(&local_values_to_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
9
10  // Every rank prints the sum
11  printf("Sum from rank %d : %4.2f \n", mpi_rank, sum);
```

The result of the reduction operation is stored in the variable `sum` on rank 0, while all other ranks have the variable `sum` unchanged.

Partial list of MPI_Reduce operations

MPI Reduction Operations

A partial list of the available reduction operations in MPI is shown in the following table.

MPI_Op	MPI_Datatype	Description
MPI_MAX	Integer, Floating-point	Maximum value
MPI_MIN	Integer, Floating-point	Minimum value
MPI_SUM	Integer, Floating-point, (Complex)	Sum
MPI_PROD	Integer, Floating-point, (Complex)	Product
MPI_LAND	Integer, Logical	Logical AND
MPI_BAND	Integer, Byte	Bitwise AND
MPI_LOR	Integer, Logical	Logical OR
MPI_BOR	Integer, Byte	Bitwise OR
MPI_LXOR	Integer, Logical	Logical XOR
MPI_BXOR	Integer, Byte	Bitwise XOR
MPI_MAXLOC	Structs (Value + Index)	Maximum value and its associated index/rank
MPI_MINLOC	Structs (Value + Index)	Minimum value and its associated index/rank
MPI_REPLACE	Any	Overwrites target with initial value (primarily used in RMA/one-sided communication)

MPI_Reduce with the MPI_IN_PLACE variable

MPI Reduction Operations

It is also possible to save the reduction operation ontop of the sendbuffer of the receiving ran. The variable MPI_IN_PLACE is used to indicate when the receiving rank receives the result of the reduction operation on the same variable (buffer) that is used for the output data.

The example of this type of reduction operation can be found in the example `4.3.MPI_Reduce_IN_PLACE.c` file.

```
1   if (mpi_rank == 0)
2   {
3      // Rank 0 passes MPI_IN_PLACE for sendbuffer; result is stored directly in '
       local_values_to_sum'
4      MPI_Reduce(MPI_IN_PLACE, &local_values_to_sum, 1, MPI_DOUBLE, MPI_SUM,
5            0, MPI_COMM_WORLD);
6   }
7   else
8   {
9      // Other ranks pass their local buffer to sendbuffer; recvbuffer is ignored (NULL)
10     MPI_Reduce(&local_values_to_sum, NULL, 1, MPI_DOUBLE, MPI_SUM,
11           0, MPI_COMM_WORLD);
12  }
```

This is used in particular when the sending ranks do not allocate the receiving buffer.

Notice that for all other processes that do not receive the result of the reduction operation, the `data_reduce` variable is set to NULL.

MPI_Allreduce

MPI Reduction Operations

MPI_Allreduce is a collective communication operation that combines the functionality of both reduction and broadcasting.

It performs a reduction operation across all processes in a communicator and then distributes the result to all processes.

```
int MPI_Allreduce(
    const void *data_to_reduce,
    void *data_reduced,
    int data_size,
    MPI_Datatype data_type,
    MPI_Op reduce_operation,
    MPI_Comm comm)
```

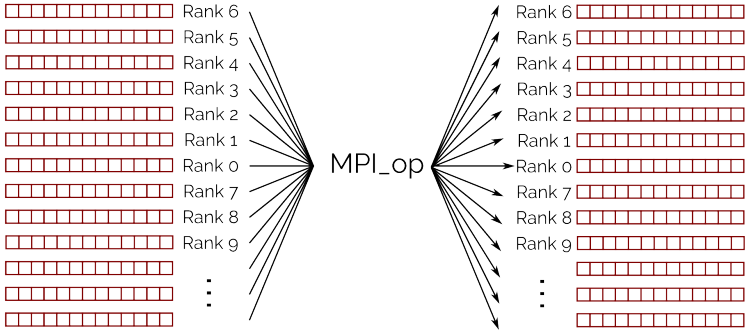
```
int MPI_Iallreduce(
    const void *data_to_reduce,
    void *data_reduced,
    int data_size,
    MPI_Datatype data_type,
    MPI_Op reduce_operation,
    MPI_Comm comm,
    MPI_Request *request)
```

Characteristics:

- Every process in the communicator (comm) must call MPI_Allreduce with matching parameters (`data_size`, `data_type`, `reduce_operation`);
- The receive buffer (`data_reduced`) variable is meaningful on all processes, as each process receives the final reduced result;
- Rather than gathering all data sequentially on process 0, MPI implementations typically uses butterfly-structured internal communications, achieving $O(\log_2 p)$ time complexity for p processes

MPI_Allreduce function

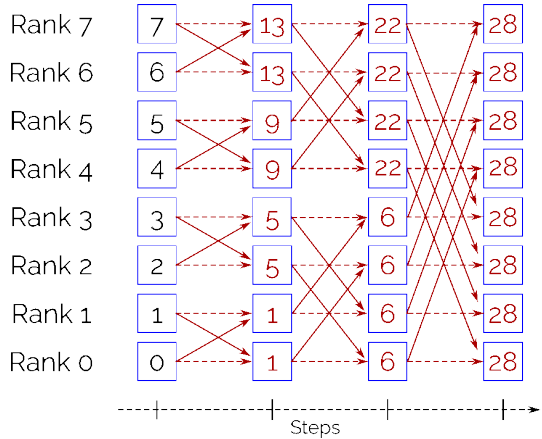
MPI Reduction Operations



Butterfly algorithms for Allreduce operations

MPI Reduction Operations

Tree (butterfly) structures for Allreduce operations the computational time of the communications to $\mathcal{O}(\log_2 p)$ where p is the number of processes.



Example for the MPI_Allreduce function

MPI Reduction Operations

An example of the usage of the MPI_Allreduce function can be found in the example `4.4.MPI_Allreduce.c` file.

```c
// All ranks initialize the values that contribute to the sum.
double local_values_to_sum = (double)mpi_rank;

// All ranks initialize the values of the sum
double sum = 0.0;

// Perform the reduction operation to sum values across all ranks
MPI_Allreduce(&local_values_to_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Also in this case we can use the MPI_IN_PLACE variable to indicate that all the processes receive the result of the reduction operation on the same buffer that is used for the output data.

```c
// Perform the reduction operation on the variables to reduce
MPI_Allreduce(MPI_IN_PLACE, &local_values_to_sum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
```

Université du Luxembourg

Exercise: Simple Reduce exercises



Parallelization with `MPI_Reduce`

Exercise: Simple Reduce exercises

Consider the two exercises on the **integration of the cosine function** and the **dot product of two vectors**, that we recall below, parallelize the operations using `MPI_Reduce` and `MPI_Allreduce`.

1. **Integrate the function cosine in the interval** $[-\pi/2, \pi/2]$.

2. **Consider two vectors** $\mathbf{a}$ **and** $\mathbf{b}$ **of** N **elements each, compute their dot product defined as**

$$c = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{N} a_i b_i.$$

(Modify the examples that you can find in the Examples/Solutions folder of my github repository github.com/mbarborini/IntroductionToMPI)